

RAUL LAB · DOCUMENTACIÓN CANÓNICA DERIVADA DE MKDOCS

# Replicant Lab

Infraestructura, hosts, red, despliegues y operación.

**Versión reproducible**

sha256:a651ec13779e8f2ea8e561d536e413bfdbfb54f3817574ab84371c2fb602a94a

**Páginas fuente**

27

**Diagramas Mermaid**

7

# Índice

[Inicio](#)[Arquitectura](#)[Evolución](#)

## HOSTS

[Replicant](#)[Nexus](#)[DigitalOcean](#)[GitHub](#)

## RED

[Visión general](#)[Inventario provisional](#)

## DESPLIEGUE

[Modelos](#)[Git](#)[Docker](#)

## OPERACIÓN

[SSH](#)[Comandos útiles](#)[Mantenimiento](#)[Descargas](#)[Pendientes](#)

## APLICACIONES

[Índice](#)[PULA](#)[ErasmusHomes · Control del MVP](#)[Autodocumentación](#)[Decisiones](#)

## CHANGE LOG

[Índice](#)[21 de agosto de 2026](#)[13 de agosto de 2026](#)[9 de agosto de 2026](#)[8 de agosto de 2026](#)

# Replicant Lab

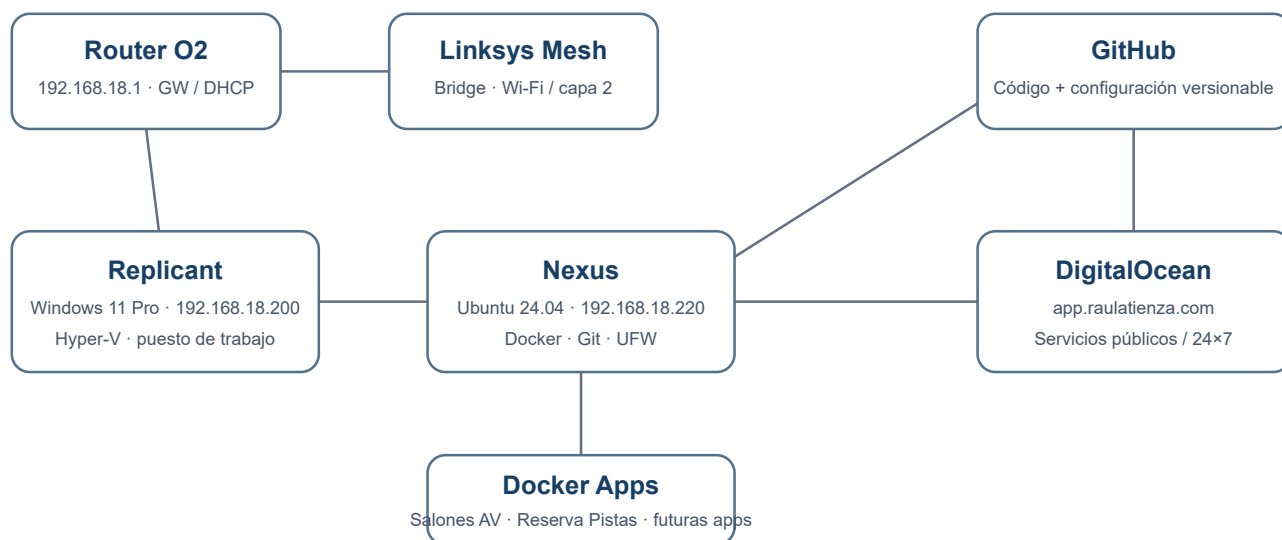
Manual vivo de la infraestructura local y cloud del laboratorio: qué existe, dónde se ejecuta, cómo se opera y qué queda pendiente.

## Objetivo

Mantener una visión única, entendible y versionada de **concepto, hosts, red, seguridad, Git, Docker y operación**. La documentación funcional de cada aplicación vive en su propio repositorio.

Las versiones portables se descargan desde el icono **↓ Descargas** de la cabecera, junto al selector claro/oscuro.

## Arquitectura de un vistazo



La portada usa **SVG nativo**, no Mermaid. Así evitamos que una incompatibilidad del parser pueda romper la vista principal.

## Principios

- **Minimalismo:** pocas piezas y cada una con una función clara.
- **Git como fuente de verdad:** código y configuración versionable viven en GitHub.
- **Separación:** Windows para trabajo interactivo; Ubuntu para servicios; cloud para disponibilidad pública/24x7.
- **Persistencia fuera de Git:** datos, secretos y backups no se mezclan con repositorios.
- **Seguridad práctica:** SSH por clave, UFW y puertos publicados de forma explícita.
- **Reproducibilidad:** un host debe poder reconstruirse sin depender de cambios manuales no documentados.
- **Separación:** Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local.

## Estado actual

| Componente | Estado      |
|------------|-------------|
| Replicant  | ✅ Operativo |

| Componente                        | Estado                                                           |
|-----------------------------------|------------------------------------------------------------------|
| Nexus                             | ✔ Operativo                                                      |
| SSH por clave                     | ✔ Operativo                                                      |
| UFW                               | ✔ Activo                                                         |
| Docker / Compose                  | ✔ Operativo                                                      |
| GitHub desde Nexus                | ✔ Operativo                                                      |
| App Launch                        | ✔ Multientorno validado · Nexus 80 + DigitalOcean HTTP/HTTPS     |
| Puerto 8080                       | ✔ Libre y no asignado                                            |
| Salones AV                        | ✔ Operativa en Nexus · 8081 · Git/Nexus reconciliados en 8c0bc08 |
| Replicant Lab en Nexus            | ✔ Runtime Nginx estático validado · 8082                         |
| Reserva-Pistas-UTP                | ✔ Validada y observada en Nexus · 8083                           |
| Cartera Estratégica               | ✔ MVP local en Replicant · no desplegada en Nexus                |
| Reserva-Pistas histórico en Nexus | ✔ 18 registros reconciliados bajo demanda · sin conflictos       |
| Consumos Cupra                    | ✔ Cloud Run · 9f66a368 · revisión 00009-pon al 100 %             |
| CV                                | ✔ Firebase Hosting · HTTP 200 · deuda POST-CARTERA               |
| Control de Red                    | ✔ Herramienta local · separación de datos POST-CARTERA           |
| Producción DigitalOcean           | ✔ App Launch y Reservas; no es runtime de Consumos ni CV         |
| Backups Nexus                     | 🕒 POST-CARTERA                                                   |
| Nombres locales                   | ✔ replicant y nexus mediante hosts en Replicant                  |

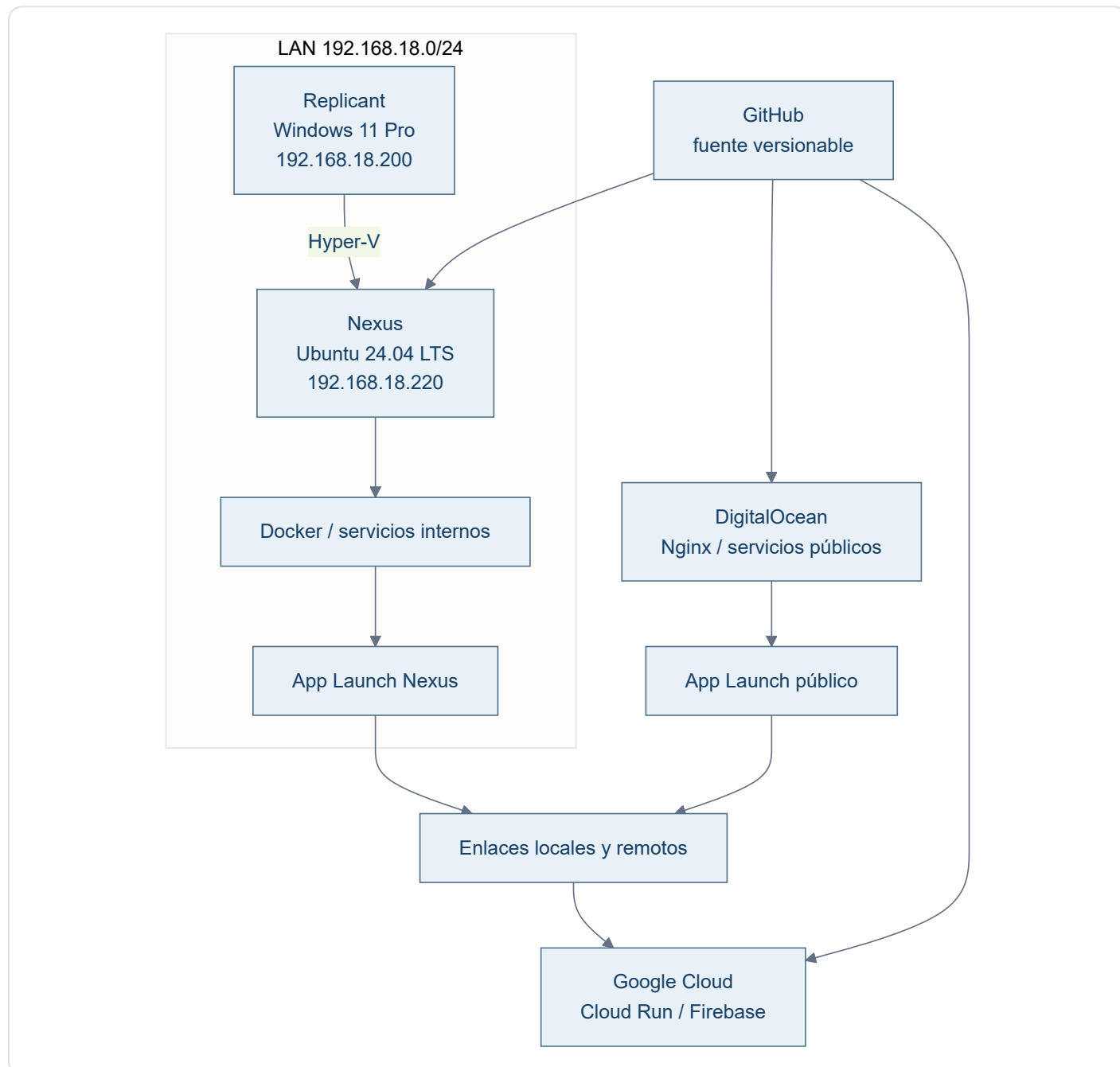
## Cómo usar esta documentación

- **Arquitectura** explica cómo encajan las piezas.
- **Evolución** resume hitos y estado actual; el detalle histórico vive en Change Log.
- **Hosts** describe cada máquina.
- **Red** documenta direccionamiento e inventario.
- **Despliegue** explica los modelos heterogéneos: local, Docker, VPS, Cloud Run y Firebase.
- **Aplicaciones** contiene solo la ficha de infraestructura de cada app.
- **Operación** concentra comandos y procedimientos cortos.
- **Pendientes** conserva el estado y los encargos que deben retomarse sin depender de memoria de chat.
- **Decisiones** registra criterios que no conviene redescubrir cada vez.
- **Descargas** ofrece el documento global y las fichas HTML/PDF derivadas de cada aplicación.

Los estados distinguen entre contenido **implementado en Git**, **probado localmente**, **validado u observado en Nexus** y **validado en producción**. Una evidencia de un entorno no se extrapola a otro.

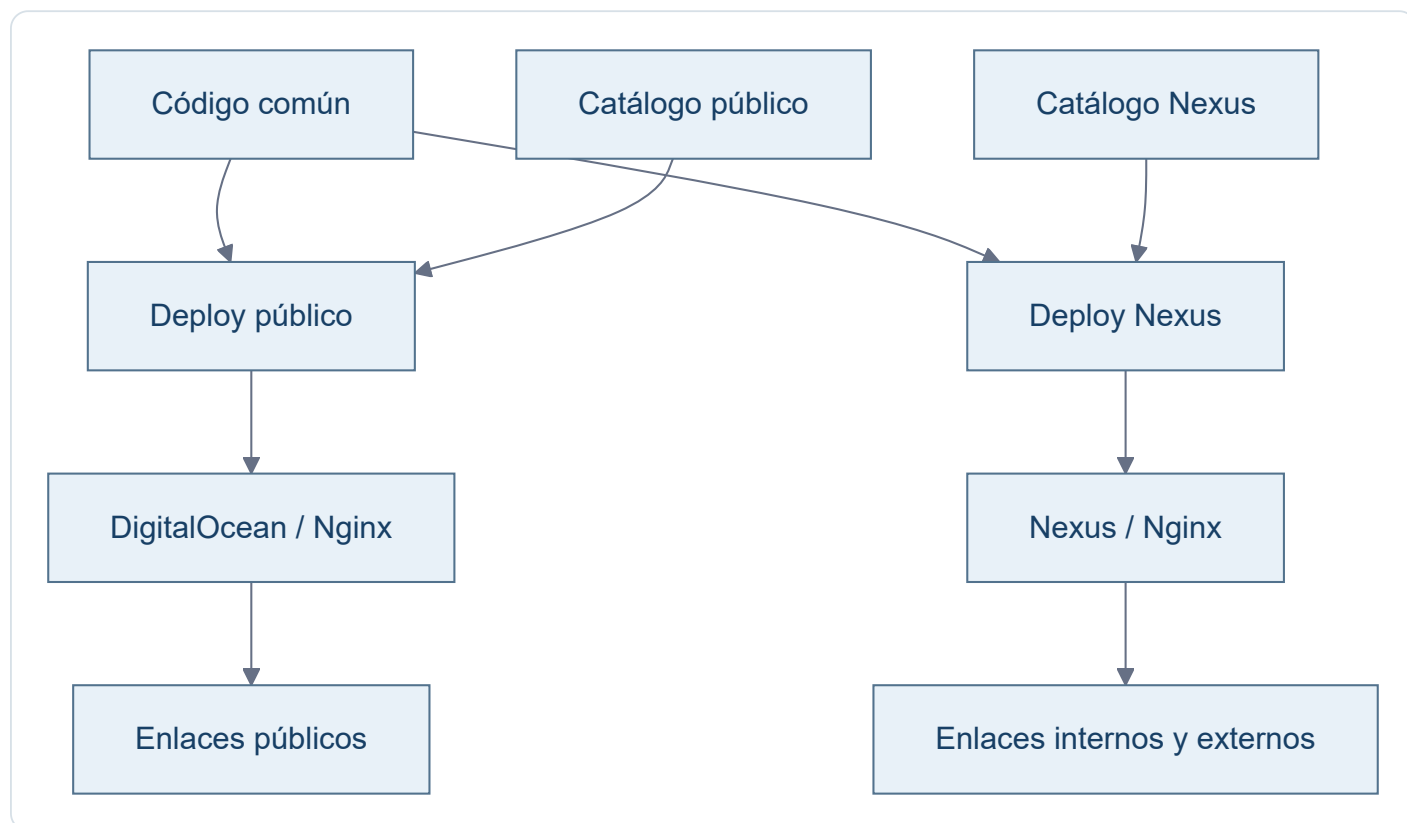
# Arquitectura

## Modelo general



App Launch es catálogo y capa de acceso. No ejecuta las aplicaciones enlazadas ni demuestra que residan en el mismo host.

## App Launch multientorno



El catálogo se selecciona durante el despliegue. Cada host recibe únicamente su `apps.json`; la lógica visual es común.

## Responsabilidades

| Elemento                                                                       | Responsabilidad                                                        |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Replicant                                                                      | Estación principal Windows, desarrollo local, Hyper-V y administración |
| Nexus                                                                          | Laboratorio Linux, Docker, servicios internos y copias de consulta     |
| GitHub                                                                         | Código, configuración versionable e histórico                          |
| DigitalOcean                                                                   | App Launch público, Reservas y otros servicios publicados en el VPS    |
| Google Cloud Run                                                               | Producción canónica de Consumos Cupra                                  |
| Firebase Hosting                                                               | Producción pública del CV                                              |
| App Launch                                                                     | Catálogo y navegación hacia aplicaciones locales o remotas             |
| <code>/opt/data</code> , <code>/opt/secrets</code> , <code>/opt/backups</code> | Datos, secretos y copias fuera de Git                                  |

### Dos reglas de lectura

Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local.

Docker es el patrón preferido para servicios internos de Nexus cuando encaja, no un requisito universal para todas las aplicaciones.

# Evolución del laboratorio

La historia detallada se conserva en el [Change Log](#). Para operar el laboratorio basta con estos hitos:

| Etapa                     | Resultado                                                                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Construcción inicial      | Replicant quedó como estación principal y host Hyper-V; Nexus como laboratorio Ubuntu/Docker.                                                     |
| Auditoría y normalización | Se inventariaron hosts, red, aplicaciones, checkouts y runtimes sin confundir presencia de código con ejecución.                                  |
| Remediaciones             | Salones AV quedó reconciliado en Git/Nexus; Consumos Cupra quedó trazado y restaurado en Cloud Run; el CV quedó identificado en Firebase Hosting. |
| Estado actual             | Las fases 2A, 2B y 2C están cerradas. No hay una incidencia crítica abierta en esas aplicaciones.                                                 |
| Trabajo futuro            | La deuda aceptada se concentra en <a href="#">Pendientes</a> y se retomará después de Cartera Estratégica.                                        |

## Criterio de validación

Cada aplicación se valida con los mecanismos apropiados para su tecnología: tests, lint, build, configuración, Compose, healthchecks, runtime o HTTP. La evidencia concreta vive en su [ficha técnica](#), sin duplicar aquí los listados de pruebas.

# Host · Replicant

| Campo          | Valor                                                           |
|----------------|-----------------------------------------------------------------|
| Host           | Replicant                                                       |
| Rol            | Estación de trabajo + host Hyper-V                              |
| SO             | Windows 11 Pro                                                  |
| IP             | 192.168.18.200                                                  |
| Gateway        | 192.168.18.1                                                    |
| Virtualización | Hyper-V                                                         |
| Switch         | Replicant Ethernet                                              |
| Acceso         | Sesión local de Windows o Escritorio remoto: mstsc /v:replicant |

## Responsabilidades

- Herramientas interactivas: ChatGPT, Codex, Gemini y similares.
- Administración de Nexus.
- Hyper-V.
- Punto de entrada SSH hacia Nexus y DigitalOcean.

## Criterio

No convertir Replicant en servidor por comodidad si el servicio puede vivir limpiamente en Nexus.



# Host · Nexus

| Campo      | Valor                                                              |
|------------|--------------------------------------------------------------------|
| Hostname   | nexus                                                              |
| SO         | Ubuntu Server 24.04.4 LTS                                          |
| IP         | 192.168.18.220/24                                                  |
| Gateway    | 192.168.18.1                                                       |
| Interfaz   | eth0                                                               |
| MAC        | 00:15:5d:12:7b:00                                                  |
| Plataforma | VM Hyper-V Gen2                                                    |
| Acceso     | ssh raul@nexus                                                     |
| Función    | Laboratorio Linux, Docker, servicios internos y copias de consulta |

## Software base

- OpenSSH Server.
- Docker Engine CE.
- Docker Compose.
- Git.
- UFW.
- Nano.
- unattended-upgrades.

## Estructura

/opt/apps repositorios y aplicaciones /opt/data datos persistentes /opt/backups copias /opt/compose composiciones separadas si hacen falta  
/opt/secrets secretos y .env fuera de Git

## Seguridad

### Estado

SSH por clave, UFW activo y actualizaciones automáticas de seguridad habilitadas.

## Puertos conocidos

| Puerto | Uso | Ámbito |
|--------|-----|--------|
| 22/tcp | SSH | LAN    |

| Puerto   | Uso                            | Ámbito       |
|----------|--------------------------------|--------------|
| 80/tcp   | App Launch                     | LAN          |
| 8080/tcp | Libre                          | Sin listener |
| 8081/tcp | Salones AV                     | LAN          |
| 8082/tcp | Replicant Lab · Nginx estático | LAN          |
| 8083/tcp | Reserva-Pistas-UTP · Nginx     | LAN          |
| 53       | systemd-resolved               | localhost    |

## Estado observado el 13/08/2026

- Docker y `unattended-upgrades` activos.
- Replicant Lab ejecutándose como sitio estático en Nginx, construido desde el `Dockerfile` y publicado en `192.168.18.220:8082` → `80`, sin bind mounts ni servidor de desarrollo.
- Salones AV accesible mediante Nginx en `192.168.18.220:8081`; checkout limpio e idéntico a GitHub `main` en `8c0bc08` después de la Fase 2A.
- Reserva-Pistas-UTP ejecutándose como backend privado y proxy Nginx en `192.168.18.220:8083`, con autenticación, datos persistentes separados y canal saliente SSH restringido hacia DigitalOcean.
- App Launch ejecutándose en el puerto `80` mediante Nginx `1.27-alpine`, con sitio y configuración montados en solo lectura.
- `8080` sin listener.
- CV y Control de Red presentes solo como checkouts; no son servicios Nexus.
- Consumos Cupra y Cartera Estratégica sin checkout servido ni puerto Nexus.
- La tarjeta PULA de App Launch apunta a la publicación pública externa y fue validada en el catálogo Nexus `2d265ee`; no implica ejecución de PULA en Nexus.

Esta observación confirma el estado del laboratorio privado en esa fecha. DigitalOcean se validó por separado y de forma acotada para el cierre de Reserva-Pistas-UTP; cada repositorio conserva sus definiciones operativas.

## Runtime documental validado en Nexus

El PR #9 fusionado sustituye `mkdocs serve` y los bind mounts por una imagen construida desde el `Dockerfile`: MkDocs estricto en la etapa builder y Nginx estático en runtime, manteniendo `192.168.18.220:8082`. El 09/08/2026 se reconstruyó y recreó exclusivamente este servicio en Nexus y se validaron HTTP, navegación, recursos, cinco diagramas Mermaid y descargas HTML/PDF idénticas byte a byte a los artefactos versionados. El HTML funciona offline sin dependencias esenciales externas y el PDF conserva la documentación completa.

# Host · DigitalOcean

| Campo     | Valor                                                                   |
|-----------|-------------------------------------------------------------------------|
| Alias SSH | docean                                                                  |
| Host      | app.raulatienza.com                                                     |
| SO        | Linux en VPS DigitalOcean                                               |
| Acceso    | Desde Replicant: <code>ssh docean</code> (SSH administrativo por clave) |
| Función   | Nginx, App Launch público y servicios web alojados en el VPS            |

## Modelo operativo

DigitalOcean publica App Launch como capa de navegación y aloja Reserva-Pistas-UTP. Debe mantener despliegues desde fuentes versionadas, secretos fuera de Git y mínima configuración manual.

App Launch puede enlazar servicios externos. En particular, Consumos Cupra se ejecuta en Google Cloud Run, el CV en Firebase Hosting y PULA en su publicación pública de Google/AI Studio; sus tarjetas no los convierten en runtimes de DigitalOcean.

## Estado validado

- App Launch respondió `200` por HTTP/HTTPS y sus assets y catálogo público fueron verificados; la tarjeta PULA apunta a la URL pública exacta y figura en el catálogo versionado `2d265ee`.
- Reserva-Pistas-UTP mantiene backend local y publicación mediante Nginx con autenticación.
- El canal de sincronización de Reservas con Nexus usa una clave dedicada y un comando SSH forzado, sin shell, TTY ni forwarding.
- No existe sincronización automática de los datos de Reservas: el operador revisa una vista previa y confirma una dirección.

La documentación no incluye contraseñas, claves privadas ni valores de secretos. Los cambios de DNS, certificados, servicios o datos requieren un encargo específico.

# Host lógico · GitHub

GitHub no es un host de ejecución del lab, pero sí una pieza de infraestructura crítica porque actúa como **fuentes de verdad**.

## Reglas

- Código y configuración versionable viven aquí.
- Cambios relevantes se realizan en rama.
- Se revisan mediante Pull Request.
- `main` representa el estado estable aprobado.
- Datos, bases de datos, `.env`, tokens y claves privadas no se versionan.

## Repos relevantes

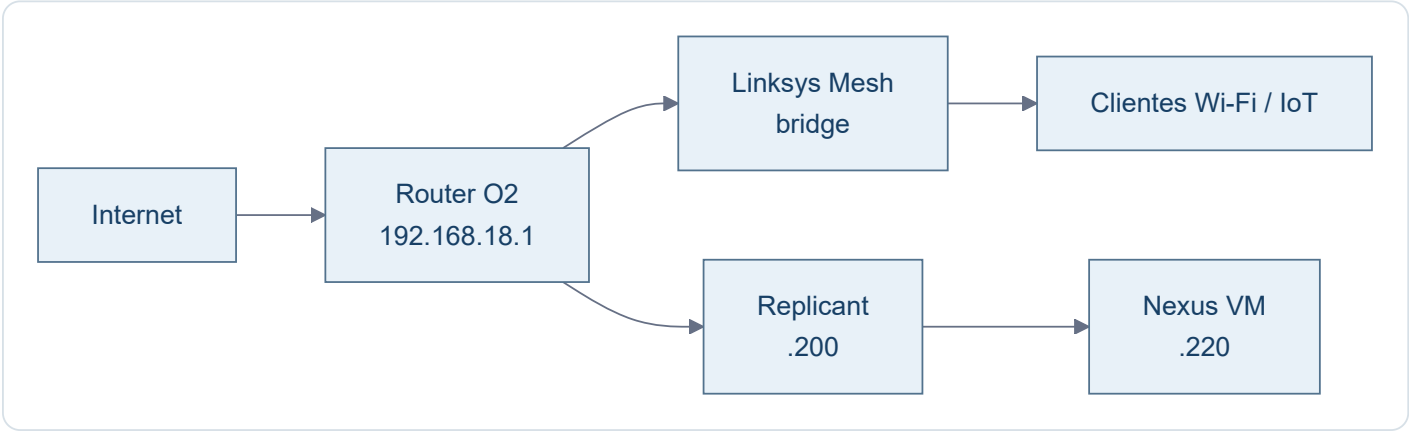
| Repo                                             | Función                                        |
|--------------------------------------------------|------------------------------------------------|
| <code>ratienza/infra-replicant-lab</code>        | Documentación viva de infraestructura          |
| <code>ratienza/salones-av-valencia-palace</code> | Aplicación AV / PoC Docker                     |
| <code>ratienza/carera-estrategica</code>         | Aplicación de cartera                          |
| <code>ratienza/Reserva-Pistas-UTP</code>         | Aplicación de reservas de pádel                |
| <code>ratienza/Apps_Launch</code>                | Catálogo público e interno de aplicaciones     |
| <code>ratienza/Consumos_Cupra</code>             | Aplicación de consumos desplegada en Cloud Run |
| <code>ratienza/CV-Raul-IA-Estudio-Google-</code> | Fuente del CV publicado en Firebase Hosting    |
| <code>ratienza/control-red</code>                | Herramienta local de inventario de red         |

Cada aplicación conserva en su propio repositorio su código, configuración reproducible y documentación funcional. Este repositorio solo mantiene la visión de infraestructura y las diferencias comprobadas entre entornos.

Un checkout no prueba que exista un runtime, y un trigger conectado a GitHub no se considera producción si la cadena real no está demostrada.

# Red · Visión general

## Topología



## Convención provisional

| Rango       | Uso previsto                   |
|-------------|--------------------------------|
| .1          | Router / gateway               |
| .2 - .19    | Infraestructura de red / mesh  |
| .20 - .179  | Clientes, IoT y DHCP general   |
| .180 - .199 | Equipos fijos, NAS, impresoras |
| .200 - .219 | Servidores físicos             |
| .220 - .239 | VMs / laboratorio              |
| .240 - .254 | Reserva futura                 |

## DNS

No existe servidor DNS local. Replicant mantiene dos aliases puntuales en el archivo `hosts` de Windows:

| Nombre    | IP             | Uso                                       |
|-----------|----------------|-------------------------------------------|
| replicant | 192.168.18.200 | Escritorio remoto y herramientas del host |
| nexus     | 192.168.18.220 | SSH y <code>http://nexus/</code>          |

El alias solo resuelve el nombre. El protocolo, puerto, usuario y credenciales siguen perteneciendo a cada servicio.

# Red · Inventario provisional

## Estado

Inventario de trabajo obtenido del escaneo de agosto de 2026. No se considera una CMDB definitiva y algunos nombres/fabricantes pueden requerir validación posterior.

## Infraestructura y equipos conocidos

| IP  | Nombre / función           | MAC / fabricante / nota              |
|-----|----------------------------|--------------------------------------|
| .1  | Router Fibra O2            | 2C-96-82-69-25-F2 · gateway          |
| .3  | RAN-CASA                   | HP                                   |
| .4  | Sonos 5                    | 5C-AA-FD-0D-54-36 · no visto         |
| .5  | Enchufe Emma               | 4C-11-AE-14-10-6E · IoT              |
| .6  | Keres.local                | 14-91-82-8D-8C-6A · Linksys          |
| .7  | SONOS Cocina               | B8-E9-37-E7-51-2E                    |
| .8  | Enchufe ordenador RAN-CASA | D4-A6-51-E5-0F-24 · Tuya             |
| .9  | Termostato Nest            | 18-B4-30-C4-52-48                    |
| .10 | Sonos Habitación           | B8-E9-37-E1-8E-AE                    |
| .11 | Echo Darío                 | F4-03-2A-7B-1B-1B                    |
| .12 | Alexa Comedor              | DC-91-BF-D1-35-B3                    |
| .13 | Linksys13497.local         | 14-91-82-8D-8C-B2 · Linksys          |
| .14 | Alexa Despacho             | 20-A1-71-00-43-44                    |
| .18 | Play Darío                 | 0C-FE-45-37-AA-4F · Sony             |
| .19 | Alexa Cocina               | 1C-FE-2B-4F-5A-1B                    |
| .20 | Fire Stick                 | 34-AF-B3-DE-58-06                    |
| .25 | IoT desconocido            | 28-6D-CD-49-AF-28 · no visto         |
| .27 | Lamparita Darío            | 28-6D-CD-56-C8-41                    |
| .35 | Lamparita comedor          | 28-6D-CD-5D-BD-81                    |
| .42 | Chromecast antiguo         | 8A-05-36-D5-F8-2D · no visto         |
| .44 | Escalera                   | 40-F5-20-C1-C7-FB · luces · no visto |
| .45 | Lamparita comedor escalera | 28-6D-CD-49-AB-D1                    |
| .52 | Enchufe Raúl               | D4-A6-51-E6-94-8B                    |
| .53 | Sebastian - CECOTEC        | 44-87-63-97-02-EA                    |

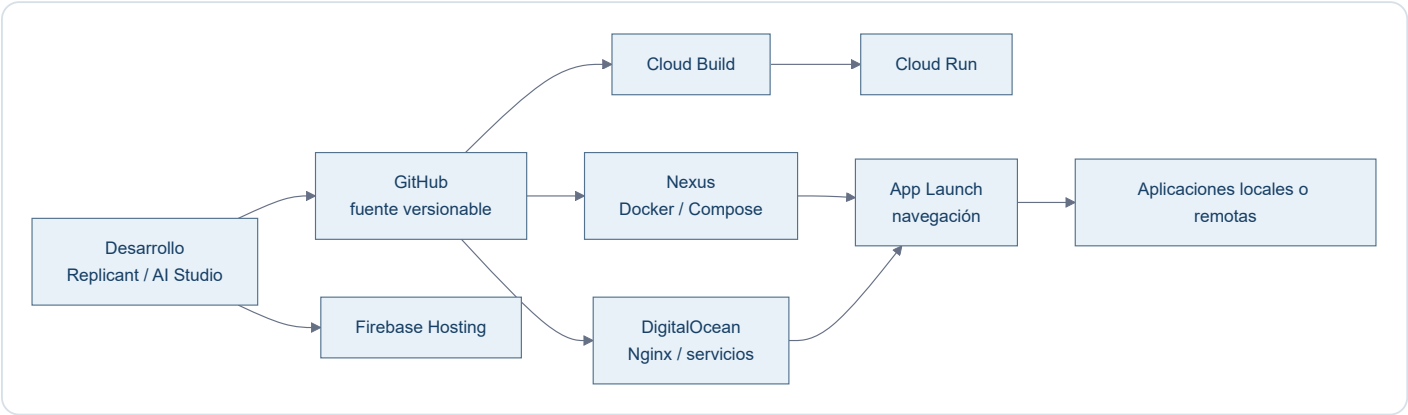
| IP   | Nombre / función             | MAC / fabricante / nota                 |
|------|------------------------------|-----------------------------------------|
| .54  | Impresora Darío              | 28-C5-C8-AB-D7-BE · HP                  |
| .71  | Luz despacho                 | CE-CC-5A-B6-E9-BC · no visto            |
| .74  | Móvil Emma                   | 76-9D-67-FE-0D-CA · Samsung · no visto  |
| .82  | Chromecast                   | F6-45-CE-C2-7C-F6                       |
| .83  | Móvil Darío                  | BA-D8-D1-F2-04-44 · no visto            |
| .94  | Portátil SH                  | 28-92-00-45-01-CD · HP · no visto       |
| .101 | Móvil Emma antiguo           | 58-02-05-B8-4E-BA · no visto            |
| .106 | Móvil Raúl                   | 52-1E-45-1D-FB-AA                       |
| .123 | Antigua IP DHCP de Replicant | 00-E0-4C-4B-35-AD · migrado a .200      |
| .124 | No identificado              | A4-E8-8D-08-12-44 · no visto            |
| .125 | Antigua IP DHCP de Nexus     | 00-15-5D-12-7B-00 · migrado a .220      |
| .200 | <b>Replicant</b>             | host físico Windows / Hyper-V · IP fija |
| .220 | <b>Nexus</b>                 | VM Ubuntu / Docker · IP fija            |

## Nota operativa

El inventario sirve para orientación y futuras decisiones de direccionamiento. No se reubicarán dispositivos por estética ni se convertirán en IP fija salvo necesidad concreta.

# Modelos de despliegue

Replicant Lab es deliberadamente heterogéneo: el runtime se elige por las necesidades de cada aplicación y no por una regla única.



| Aplicación          | Creación / desarrollo      | Deploy                                          | Runtime                       |
|---------------------|----------------------------|-------------------------------------------------|-------------------------------|
| Salones AV          | Codex / GitHub             | Docker Compose                                  | Nexus                         |
| Reserva-Pistas-UTP  | Codex / GitHub             | Compose en Nexus y despliegue controlado en VPS | Nexus + DigitalOcean          |
| Consumos Cupra      | AI Studio + Codex / GitHub | Cloud Build + Artifact Registry                 | Google Cloud Run              |
| CV                  | AI Studio / GitHub         | Despliegue manual observado                     | Firebase Hosting              |
| Cartera Estratégica | Codex / desarrollo local   | Ejecución local                                 | Replicant                     |
| Control de Red      | PowerShell + Codex         | Ejecución local                                 | Replicant                     |
| App Launch          | Codex / GitHub             | Scripts por destino                             | Nginx en Nexus y DigitalOcean |
| Replicant Lab       | Codex / GitHub             | Docker Compose                                  | Nexus                         |

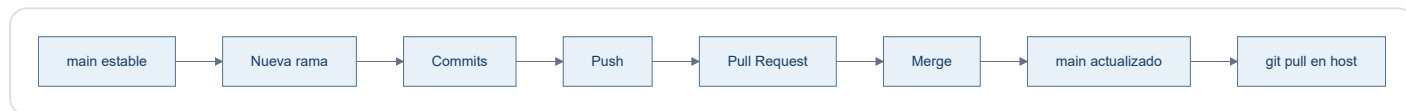
## Principios

- Checkout ≠ Runtime : un repositorio presente en un host puede ser solo una copia de consulta.
- Tarjeta App Launch ≠ Runtime local : una tarjeta es un enlace y no demuestra dónde se ejecuta la aplicación.
- App Launch es una capa de navegación; la autenticación, los datos y el ciclo de vida pertenecen a cada aplicación.
- Docker es el patrón principal en Nexus, no el único modelo del laboratorio.



# Despliegue · Git

## Flujo normal



## Convención de ramas para esta documentación

Las ramas representan **cambios lógicos**, no archivos individuales.

Ejemplos:

- docs/bootstrap-infra
- docs/app-salones
- docs/app-cartera
- docs/network
- docs/backups

## Regla

### Main

main debe representar siempre la documentación aprobada y coherente con el estado real.

# Despliegue · Docker

Este patrón describe los servicios Docker de Nexus. No se extrapola a Cloud Run, Firebase Hosting, servicios del VPS ni aplicaciones locales; la comparación completa está en [Modelos de despliegue](#).

## Patrón

```
GitHub ↓ /opt/apps/<repo> ↓ compose.yml ↓ docker compose up -d ↓ servicios
```

## Buenas prácticas del lab

- Preferir imágenes oficiales cuando sea razonable.
- Minimizar paquetes instalados en el host.
- Persistencia fuera del contenedor.
- Secretos fuera de Git.
- Publicar solo puertos necesarios.
- Cuando proceda, ligar puertos a `192.168.18.220` en vez de `0.0.0.0`.
- Separar proxy, backend y datos cuando cada pieza tenga una responsabilidad clara.
- Ejecutar los procesos de aplicación con un usuario no root siempre que sea viable.
- Usar `restart: unless-stopped` para servicios que deben recuperarse tras reiniciar Nexus.
- No añadir paneles de administración si no resuelven un problema real.

## Convención de puertos en Nexus

| Puerto | Servicio                      | Estado                |
|--------|-------------------------------|-----------------------|
| 80     | App Launch Nexus              | Operativo             |
| 8080   | Sin asignar                   | Libre                 |
| 8081   | Salones AV                    | Operativo             |
| 8082   | Replicant Lab · documentación | Operativo             |
| 8083   | Reserva-Pistas-UTP            | Operativo             |
| 8084+  | Próximos servicios            | Asignación secuencial |

### Reglas:

- App Launch usa el puerto estándar `80`, sin puerto explícito en la URL.
- `8080` queda libre y no se reserva para App Launch.
- A partir de `8081`, los servicios se numeran consecutivamente salvo necesidad técnica justificada.
- Cada nuevo servicio debe quedar documentado aquí al asignar su puerto.
- Siempre que sea posible, publicar el servicio ligado a `192.168.18.220` y no a `0.0.0.0`.

App Launch es la excepción deliberada: publica `80` en todas las interfaces de Nexus para actuar como entrada de la LAN. No se publica por HTTPS.

Salones AV aplica explícitamente la regla LAN mediante `192.168.18.220:8081:80`. El PR `salones-av-valencia-palace#2` incorporó el bind a `main`; desde `8c0bc08` el checkout Nexus está limpio y la configuración es reproducible desde GitHub.

## Reserva-Pistas-UTP · patrón validado

Reserva-Pistas utiliza un Compose de dos servicios:

```
LAN :8083 ↓ nginx ↓ red privada Compose ↓ app:8765
```

El backend no publica `8765` hacia el host. Nginx lo alcanza mediante el DNS interno de Docker usando el nombre de servicio `app`.

Los datos se desacoplan del ciclo de vida del contenedor mediante:

```
/opt/data/reserva-pistas:/app/data
```

La aplicación recibe por entorno:

```
APP_HOST=0.0.0.0 APP_PORT=8765 APP_DATA_DIR=/app/data
```

El código conserva valores por defecto compatibles con despliegues no Docker.

## Operación estándar

```
cd /opt/apps/<proyecto> git switch main git pull docker compose up -d --build
```

Comprobaciones habituales:

```
docker compose ps docker compose logs --tail 50
```

Parada y recreación:

```
docker compose down docker compose up -d
```

Un `down` elimina contenedores y la red del proyecto, pero no debe eliminar datos persistentes alojados fuera del contenedor.

## Autoarranque

Docker Engine arranca con Ubuntu. Los contenedores existentes con `restart: unless-stopped` se recuperan automáticamente después de reiniciar Nexus.

Compose no necesita ejecutarse manualmente durante el arranque para recuperar esos contenedores ya creados; su fichero YAML sigue siendo la definición reproducible para recrearlos o actualizarlos.

## Replicant Lab · sitio estático reproducible

La definición versionada converge en un único modelo:

```
mkdocs.yml + docs/ ↓ Dockerfile · MkDocs build --strict ↓ sitio estático ↓ Nginx :80 ↓ 192.168.18.220:8082
```

`compose.yml` construye el `Dockerfile`, publica únicamente `192.168.18.220:8082 → 80` y no monta fuentes ni ejecuta `mkdocs serve`.

## Actualización desplegada

```
cd /opt/apps/infra-replicant-lab git switch main git pull --ff-only origin main docker compose up -d --build
```

Cada cambio documental desplegado requiere reconstruir la imagen porque Nginx sirve la copia estática incluida durante el build. Las dependencias viven en las etapas versionadas; no se instalan en Nexus.

## Separación de estados

El modelo estático está implementado, probado localmente y validado en Nexus. La validación del 09/08/2026 incluyó el contenedor Nginx estable, HTTP y navegación, cinco diagramas Mermaid, descargas reales idénticas a Git, HTML offline y PDF completo. No implica validación en producción.

# Operación · SSH

## Desde Replicant

```
ssh raul@nexus ssh docean
```

`ssh raul@nexus` es la referencia operativa principal. El alias corto puede depender de la configuración SSH local y no debe darse por supuesto.

## Configuración conceptual

```
Host nexus HostName 192.168.18.220 User raul IdentityFile C:\Users\raul\.ssh\nexus_local IdentitiesOnly yes Host docean HostName app.raulatienza.com User root IdentityFile C:\Users\raul\.ssh\reserva_pistas_do IdentitiesOnly yes
```

## Claves

- `nexus_local` : Replicant → Nexus.
- `reserva_pistas_do` : Replicant → DigitalOcean.
- `~/.ssh/id_ed25519` en Nexus: Nexus → GitHub.

### Nunca documentar

No almacenar en este repo claves privadas, tokens, contraseñas ni el contenido de secretos.

# Operación · Comandos útiles

## Host

```
ip addr ip route sudo ss -tulpn
```

## Firewall

```
sudo ufw status verbose
```

## Docker

```
docker ps docker compose up -d docker compose down docker compose logs
```

## Git

```
git status git switch main git pull
```

## Actualizaciones

```
sudo apt update apt list --upgradable sudo apt upgrade
```

## Filosofía

Este documento evita convertirse en un recetario infinito. Solo se incluyen comandos frecuentes y útiles para operar el lab.

# Operación · Mantenimiento

## Actualizaciones

`unattended-upgrades` está activo y habilitado. Ubuntu puede diferir ciertos paquetes mediante phased rollout; no se fuerzan salvo necesidad.

Comprobación:

```
systemctl status unattended-upgrades --no-pager
```

## Limpieza

`apt autoremove` puede utilizarse tras revisar qué paquetes se eliminarán.

## Criterio de mantenimiento

- Actualizar con regularidad.
- No instalar herramientas "por si acaso".
- Revisar servicios expuestos con `ss -tulpn`.
- Mantener `main` coherente con la realidad.

# Descargas

La documentación MkDocs de este proyecto es la única referencia canónica. Los dos ficheros siguientes se generan exclusivamente desde `mkdocs.yml` , las páginas de `docs/` incluidas en `nav` y sus recursos versionados.

↓ Descargar documentación HTML offline

↓ Descargar documentación PDF

## Rutas canónicas únicas

| Artefacto          | Ruta                                           | Cobertura                                                           |
|--------------------|------------------------------------------------|---------------------------------------------------------------------|
| HTML autocontenido | <code>docs/downloads/Replicant-Lab.html</code> | Toda la navegación MkDocs, índice interno y siete Mermaid           |
| PDF A4             | <code>docs/downloads/Replicant-Lab.pdf</code>  | Portada, índice, documentación completa, numeración y siete Mermaid |

No se mantiene ninguna copia alternativa.

## Fichas técnicas individuales

Cada pareja HTML/PDF se genera desde la misma página Markdown incluida en la navegación. El HTML funciona offline y el PDF está preparado para consulta o archivo.

| Aplicación          | HTML                      | PDF                       |
|---------------------|---------------------------|---------------------------|
| PULA                | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| App Launch          | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| Salones AV          | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| Reserva-Pistas-UTP  | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| Consumos Cupra      | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| CV de Raúl          | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| Control de Red      | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| Cartera Estratégica | <a href="#">Descargar</a> | <a href="#">Descargar</a> |
| Replicant Lab       | <a href="#">Descargar</a> | <a href="#">Descargar</a> |

## Propiedades verificables

- Todos los artefactos muestran la misma huella SHA-256 de las fuentes.
- El HTML incorpora CSS, JavaScript y Mermaid localmente y funciona mediante `file://` sin red.
- El HTML no solicita CDN ni contiene clientes de recarga, conexiones dinámicas o referencias al servidor de desarrollo.
- El PDF mantiene texto seleccionable, enlaces, tablas, código, avisos y diagramas.
- El JavaScript del sitio calcula las rutas desde su propio recurso y funciona con el sitio estático en `/`.



## Regeneración y sincronía

Después de preparar las dependencias fijadas, un único comando regenera y valida todo:

```
python scripts/docs_pipeline.py generate
```

CI ejecuta el modo `check`, vuelve a renderizar un PDF de control, verifica la huella y cobertura, construye la imagen Docker final, arranca Nginx temporalmente y compara byte a byte las descargas servidas con los artefactos versionados.

## Estado de despliegue

El pipeline y el runtime estático están implementados y probados localmente. El 21/08/2026 se comprobó el dossier global y las nueve parejas de fichas individuales, incluida PULA; los ficheros servidos por el entorno de validación coincidieron byte a byte con los artefactos generados. La publicación en Nexus se valida de nuevo después de integrar el cambio en `main`; esta evidencia corresponde al laboratorio privado y no se extrapola a producción.

## Pendientes

No hay incidencias críticas abiertas en Salones AV, Consumos Cupra o CV. Las fases 2A, 2B y 2C están cerradas.

### Activos

- Continuar Cartera Estratégica como siguiente bloque de producto.

## PULA

- Crítico antes de cualquier exposición adicional:** autenticar y autorizar la API, bloquear SSRF y sacar contactos y estado vivo de Git.
- Importante antes de trasladar a DigitalOcean:** definir persistencia, imagen y despliegue reproducibles, secretos, healthcheck, backup/restore y rollback.
- Calidad:** corregir el falso estado `sent`, añadir tests aislados y CI, fijar runtime y gestor de dependencias y reconciliar `AGENTS.md` con las funciones ya presentes.
- Mejora:** normalizar extracción, traducción y capacidad de los anuncios SCPU y añadir observabilidad sin datos personales.

PULA está documentada, pero estos puntos son deuda de la aplicación: no están implementados ni validados en Nexus o producción.

### Deuda POST-CARTERA

## CV / Firebase

- Reconciliar Firebase y Cloud Build.
- Retirar o corregir el trigger y la configuración obsoletos.
- Demostrar trazabilidad entre SHA Git y versión Firebase.
- Incorporar lockfile y build reproducible, CI, tests y typecheck.
- Revisar restos React, metadata Gemini y artefactos residuales.
- Revisar cabeceras de seguridad, caché, protección de `main` y UX de revelado de contacto.

## Control de Red

- Hacer backup antes de intervenir.
- Separar inventarios, snapshots y datos operativos reales del código Git.
- Mantener únicamente ejemplos anonimizados en el repositorio.

## Nexus

- Definir una política global de backups y ejecutar una prueba de restauración.
- Inventariar y gestionar los checkouts que son solo copias de consulta.
- Mantener explícito `Checkout ≠ Runtime`; el CV en Nexus no es un despliegue.

### Mejoras opcionales

- Limpiar assets PWA históricos no utilizados de App Launch mediante un cambio versionado y verificable.

La deuda aceptada no debe presentarse como fallo de producción ni reabrir las fases ya cerradas.

# Aplicaciones

Catálogo operativo del laboratorio. Cada ficha técnica se abre como documento HTML independiente y autocontenido.

## PULA

### BÚSQUEDA DE ALOJAMIENTO ESTUDIANTEL

POC privada para localizar y gestionar apartamentos publicados por SCPU en Pula, Croacia. Incorpora scraping, importación asistida y seguimiento Gmail; no incluye el concepto independiente ErasmusHomes.

|             |                                               |
|-------------|-----------------------------------------------|
| Herramienta | AI Studio + AntigraVity                       |
| Stack       | React, Vite, Express, Gemini                  |
| Repositorio | ratienza/Pula                                 |
| Deploy      | Producción pública existente                  |
| Runtime     | Google · no modificado                        |
| Estado      | POC beta Pula / publicada en ambos App Launch |
| URL         | <a href="#">Producción</a>                    |

[Abrir PULA](#) · [Consultar en el gestor](#)

[Ver ficha técnica](#)

## App Launch

### PORTAL MULTIENTORNO

Catálogo de acceso a aplicaciones públicas e internas. Se publica en Nexus y DigitalOcean con catálogos distintos por entorno; las tarjetas navegan hacia servicios, pero no constituyen su runtime.

|             |                              |
|-------------|------------------------------|
| Herramienta | Codex                        |
| Stack       | HTML, CSS, JavaScript        |
| Repositorio | ratienza/Apps_Lauch          |
| Deploy      | Scripts por destino          |
| Runtime     | Nginx · Nexus y DigitalOcean |
| Estado      | Operativo                    |
| URL         | <a href="#">Nexus</a>        |

[Ver ficha técnica](#)

## ErasmusHomes · Control del MVP

### CONSOLA INTERNA DE DIRECCIÓN

Panel derivado del roadmap canónico de ErasmusHomes. Resume progreso, prioridades, gates, validación de fin de semana, riesgos, evidencias y sincronización con el SHA de GitHub sin convertirse en fuente paralela.

|                    |                               |
|--------------------|-------------------------------|
| Herramienta        | Codex                         |
| Stack              | MkDocs + YAML generado        |
| Repositorio fuente | ratienza/ErasmusHomes         |
| Repositorio visual | ratienza/infra-replicant-lab  |
| Deploy             | Documentación existente       |
| Runtime            | Nexus · 8082                  |
| Estado             | Interno / EH-002              |
| URL                | <a href="#">Panel interno</a> |

[Abrir panel de control](#)

[Ver ficha técnica](#)

## Salones AV

### GUÍA OPERATIVA

Guía audiovisual para los salones del Valencia Palace. Reúne procedimientos, conexiones, sonido y mapas de apoyo para la operación diaria.

|             |                                     |
|-------------|-------------------------------------|
| Herramienta | Codex                               |
| Stack       | HTML, Nginx                         |
| Repositorio | ratienza/salones-av-valencia-palace |
| Deploy      | Docker Compose                      |
| Runtime     | Nexus · 8081                        |
| Estado      | Cerrado / operativo                 |
| URL         | <a href="#">Nexus</a>               |

[Ver ficha técnica](#)

## Reserva-Pistas-UTP

### RESERVAS Y AUTOMATIZACIÓN

Gestiona reservas de pistas de pádel y su programación. Ofrece interfaz web y bot de Telegram para consultar, crear o cancelar operaciones autorizadas.

|             |                             |
|-------------|-----------------------------|
| Herramienta | Codex                       |
| Stack       | Python, Flask, Nginx        |
| Repositorio | ratienza/Reserva-Pistas-UTP |
| Deploy      | Compose + systemd/Nginx     |
| Runtime     | Nexus y DigitalOcean        |
| Estado      | Operativo                   |
| URL         | <a href="#">Producción</a>  |

[Ver ficha técnica](#)

## Consumos Cupra

### REGISTRO Y ANÁLISIS

Registra y analiza consumos del vehículo, histórico, pendientes y estadísticas. Mantiene la información operativa desde una interfaz web conectada con servicios Google.

|             |                         |
|-------------|-------------------------|
| Herramienta | AI Studio + Codex       |
| Stack       | React, Vite, Express    |
| Repositorio | ratienza/Consumos_Cupra |
| Deploy      | Cloud Build             |
| Runtime     | Google Cloud Run        |
| Estado      | Cerrado / operativo     |

[Ver ficha técnica](#)

## CV de Raúl

### PERFIL PROFESIONAL

Presenta el currículum y portfolio profesional público. Organiza experiencia, formación y contacto, con una versión descargable en PDF.

|             |                                                  |
|-------------|--------------------------------------------------|
| Herramienta | AI Studio                                        |
| Stack       | Vite, Tailwind, PDFKit                           |
| Repositorio | <code>ratienza/CV-Raul-IA-Estudio-Google-</code> |
| Deploy      | Firebase Hosting                                 |
| Runtime     | Firebase                                         |
| Estado      | Operativo / POST-CARTERA                         |
| URL         | <a href="#">Producción</a>                       |

[Ver ficha técnica](#)

## Control de Red

### INVENTARIO LOCAL

Descubre e inventaría dispositivos de la red local. Ayuda a revisar direccionamiento, nombres y estado observable desde Replicant.

|             |                                   |
|-------------|-----------------------------------|
| Herramienta | PowerShell + Codex                |
| Stack       | PowerShell                        |
| Repositorio | <code>ratienza/control-red</code> |
| Deploy      | Ejecución local                   |
| Runtime     | Replicant                         |
| Estado      | Operativo local / POST-CARTERA    |

[Ver ficha técnica](#)

## Cartera Estratégica

### ANÁLISIS FINANCIERO

Gestiona y analiza una cartera personal de inversión. Centraliza posiciones, histórico y métricas de seguimiento en una interfaz local.

|             |                                           |
|-------------|-------------------------------------------|
| Herramienta | Codex                                     |
| Stack       | Python, Streamlit, SQLite                 |
| Repositorio | <code>ratienza/cartera-estrategica</code> |
| Deploy      | Ejecución local                           |
| Runtime     | Replicant                                 |
| Estado      | MVP operativo                             |

[Ver ficha técnica](#)

## Replicant Lab

### DOCUMENTACIÓN OPERATIVA

Documenta infraestructura, despliegue y operación del laboratorio. Genera un sitio navegable y versiones portables HTML/PDF desde fuentes Markdown canónicas.

|             |                                           |
|-------------|-------------------------------------------|
| Herramienta | Codex                                     |
| Stack       | MkDocs, Mermaid, Nginx                    |
| Repositorio | <code>ratienza/infra-replicant-lab</code> |
| Deploy      | Docker Compose                            |
| Runtime     | Nexus · 8082                              |
| Estado      | Operativo                                 |
| URL         | <a href="#">Nexus</a>                     |

[Ver ficha técnica](#)

`Checkout ≠ Runtime` y `Tarjeta App Launch ≠ Runtime local`.

# PULA

Auditoría técnica y manual funcional de `ratienza/Pula` . **PULA es exclusivamente una POC privada para localizar y gestionar apartamentos de la web oficial SCPU en Pula, Croacia.** Su finalidad actual es ayudar en una búsqueda familiar concreta; no es ErasmusHomes ni una plataforma paneuropea.

### Alcance de este documento

PULA es una POC privada de búsqueda de apartamentos SCPU en Pula, Croacia. No forma parte de ErasmusHomes.

## Estado comprobado

| Elemento                   | Evidencia                                                                                                                   |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Repositorio                | <code>ratienza/Pula</code> · privado                                                                                        |
| <code>main</code> auditado | <code>ff8165cf7c9d1d4c5ae670c56486c25900a5754b</code>                                                                       |
| Alcance funcional          | Búsqueda SCPU, gestión de candidatos, contacto y seguimiento Gmail                                                          |
| Checkout Nexus             | <code>/opt/apps/Pula</code> · limpio y sincronizado; copia de consulta, no runtime                                          |
| Producción pública         | <code>https://pula-erasmus-housing-automator.ai.studio/</code> · HTTP 200 validado el 21/08/2026; el runtime no se modificó |
| App Launch                 | Tarjeta publicada y validada en Nexus y DigitalOcean desde <code>ratienza/Apps_Lauch</code> ( <code>2d265ee</code> )        |
| Pruebas locales            | TypeScript y build Vite/esbuild correctos sobre copia temporal                                                              |
| Tests automatizados        | No existe suite; solo scripts exploratorios de scraping/Firebase                                                            |
| Datos versionados          | <code>data.json</code> : 29 registros <code>new</code> con 29 direcciones de contacto                                       |

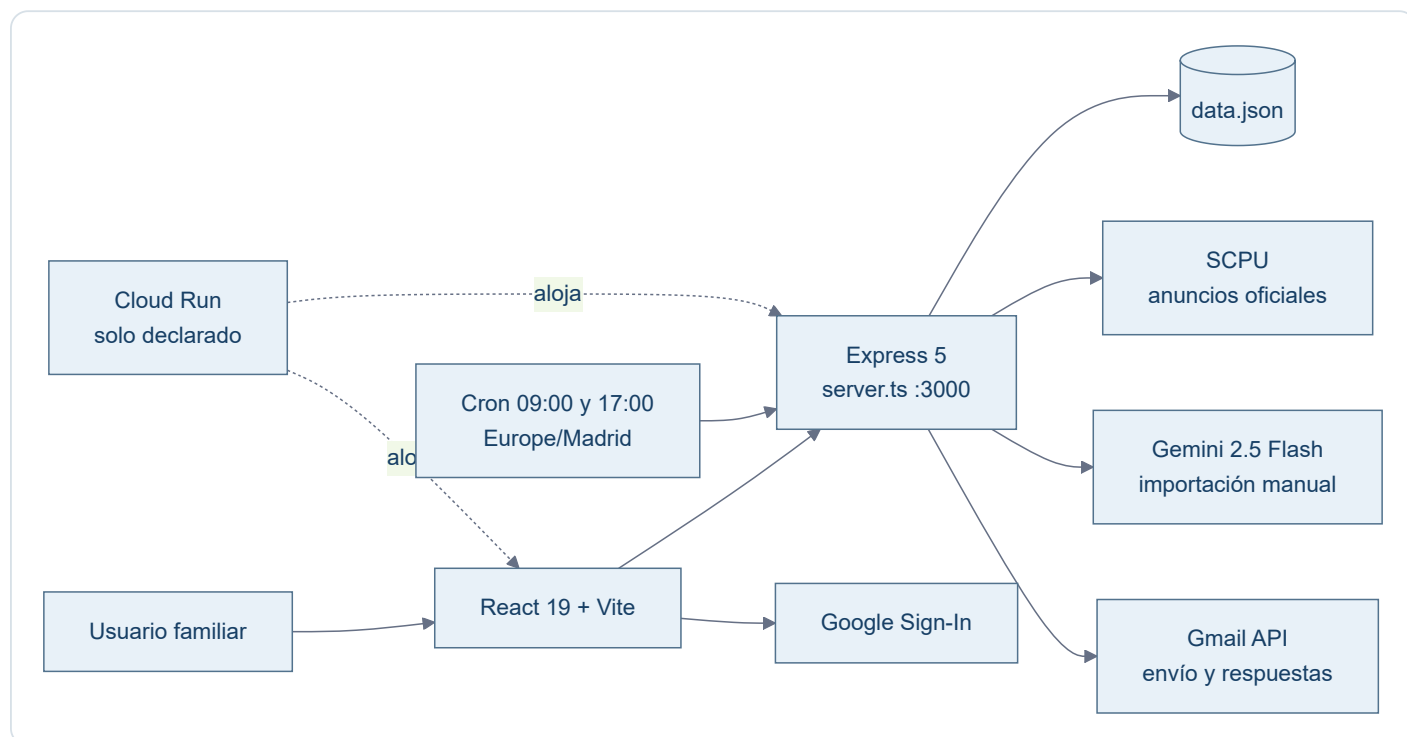
Calidad general: **5/10**. La POC compila y concentra el flujo funcional para Pula, pero carece de tests, control de acceso, persistencia robusta y trazabilidad reproducible de Cloud Run.

## Separación obligatoria de proyectos

`ratienza/ErasmusHomes` es otro repositorio y otro proyecto. Contiene el concepto comercial paneuropeo, el ejemplo visual de Bolonia/UNIBO, Smart Deposit Audit y el dossier ReportLab. Actualmente debe tratarse como **concepto no validado**, no como funcionalidad implementada de PULA.

La etiqueta `v0.1-poc-beta` y los artefactos ErasmusHomes presentes en el historial de `ratienza/Pula` son contaminación histórica y deuda de higiene del repositorio. No definen el alcance de PULA y no se usan como fuente funcional en esta ficha. Cualquier limpieza futura de esa historia o etiqueta requiere un encargo específico sobre el repositorio PULA.

## Arquitectura de `main`



El build ejecuta `vite build` y empaqueta `server.ts` con esbuild como `dist/server.cjs`. El repositorio no contiene Dockerfile, manifiesto Cloud Run, workflow CI ni definición de volumen persistente; Git no permite reproducir ni vincular el despliegue declarado con el SHA auditado.

## Modelo de datos

Cada apartamento registra UUID, casero, superficie, capacidad, precio, ubicación, descripción, email, enlace, condiciones, contrato, estado y fecha. Puede añadir origen manual y datos de respuesta. El enlace funciona como clave de deduplicación.

Estados: `new`, `sent`, `following` y `discarded`.

## Manual funcional de `main`

### Descubrimiento automático

1. **Extraer Nuevos Pisos SCPU** o el cron inicia el proceso.
2. El servidor recorre hasta cinco páginas y limita el lote a 40 enlaces.
3. Cheerio extrae casero, email, precio, superficie, zona y contrato.
4. Se traducen expresiones croatas mediante sustituciones locales.
5. Se excluyen contratos detectados como anuales y se deduplica por URL.
6. El resultado se guarda y aparece en **Nuevas Oportunidades**.

La implementación contradice la afirmación documental “sin límite artificial”: hay un máximo de cinco páginas y 40 enlaces. Además, el scraper fija la capacidad como `Ver descripción`, aunque la regla de interfaz exige un número de personas.

### Importación manual con IA

1. **Add URL** 🖱️ abre el formulario.
2. El backend descarga la URL y elimina etiquetas ruidosas.
3. Envía hasta 12.000 caracteres a Gemini 2.5 Flash con esquema JSON.
4. Si Gemini no está disponible, aplica una heurística.
5. Marca la ficha como manual y deduplica por URL exacta.

No es segura para exposición pública hasta restringir protocolos, DNS/IP, redes privadas, redirecciones, tamaño y tiempo: hoy una petición anónima puede ordenar al servidor descargar una URL arbitraria, creando riesgo de SSRF.

## Contacto y seguimiento Gmail

1. Firebase Google Sign-In entrega un access token conservado en memoria.
2. El backend construye un correo bilingüe inglés/croata y llama a Gmail API.
3. La comprobación revisa hasta 25 hilos y compara remitentes con caseros.
4. Una respuesta mueve un anuncio enviado a `following`.

Existe un defecto de integridad: el endpoint responde `success` y cambia el estado a `sent` incluso si falta un token utilizable o Gmail devuelve un error. “Enviado” no prueba que el correo haya salido.

## Gestión visual

La SPA ofrece Nuevas Oportunidades, En Seguimiento, Enviados sin respuesta e Histórico, orden por fecha o precio y cambio manual de estado. Los botones de producto están presentes. `package.json` usa React 19, aunque la arquitectura escrita declara React 18.

## Material ErasmusHomes excluido

El dossier de Bolonia, los mockups UNIBO, el backend FastAPI de auditoría de fianzas y los generadores ReportLab se excluyen expresamente de la arquitectura, API, puntuación y manual funcional de PULA. Solo se comprobó que su ubicación canónica actual es el repositorio independiente `ratienza/ErasmusHomes`; no se auditó su funcionamiento.

## API REST observada

| Método y ruta                                | Entrada                                     | Salida y efecto                                                                     | Autenticación observada | Riesgo principal                                            |
|----------------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------|-------------------------|-------------------------------------------------------------|
| GET <code>/api/config</code>                 | Ninguna                                     | JSON con <code>clientId</code> OAuth                                                | No                      | Expone configuración cliente; no es un secreto por sí misma |
| GET <code>/api/apartments</code>             | Ninguna                                     | Devuelve todos los registros                                                        | No                      | Expone contactos y datos operativos sin autorización        |
| POST <code>/api/apartments/scrape</code>     | Ninguna                                     | Consulta SCPU, deduplica y reescribe <code>data.json</code>                         | No                      | Escritura anónima, sin timeout ni control de concurrencia   |
| POST <code>/api/apartments/manual-url</code> | JSON <code>{url}</code>                     | Descarga, analiza y añade un candidato                                              | No                      | SSRF, consumo abusivo y persistencia anónima                |
| POST <code>/api/gmail/check-replies</code>   | Bearer OAuth de Google                      | Lee hasta 25 hilos y mueve respuestas de <code>sent</code> a <code>following</code> | Bearer requerido        | Token delegado y coincidencias heurísticas de remitente     |
| PUT <code>/api/apartments/{id}/status</code> | JSON <code>{status}</code>                  | Cambia el estado y reescribe el fichero                                             | No                      | Mutación anónima y posibles escrituras perdidas             |
| POST <code>/api/apartments/{id}/send</code>  | Bearer opcional; email alternativo opcional | Intenta enviar correo y cambia a <code>sent</code>                                  | Opcional en la práctica | Puede declarar éxito aunque Gmail no haya enviado           |

No se observan OpenAPI versionado, rate limiting, sesión de aplicación, autorización por usuario ni validación estructural centralizada.

## Datos y persistencia

`data.json` es simultáneamente semilla versionada y base de datos de ejecución. El SHA auditado contiene 29 apartamentos en estado `new` y 29 direcciones de contacto. Cada registro puede incluir identidad del casero, email, enlace, precio, ubicación, descripción, condiciones, contrato, estado, origen manual y metadatos de respuesta.

- **Persistencia implementada:** lectura y escritura síncronas del JSON completo mediante `fs.readFileSync` y `fs.writeFileSync`.
- **Concurrencia:** no hay bloqueo, control de versión ni reemplazo atómico; dos peticiones pueden sobrescribirse.
- **Cloud Run:** el filesystem del contenedor no constituye almacenamiento duradero; Git no define volumen ni servicio externo.
- **Privacidad:** los emails son datos personales y no deberían permanecer en Git ni exponerse por una API anónima.
- **Recuperación:** no existe procedimiento versionado de backup, restauración o migración.

## Seguridad

| Severidad | Hallazgo confirmado                             | Evidencia                                                                                         | Recomendación futura                                                                                      |
|-----------|-------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Crítica   | API de lectura y mutación sin control de acceso | CORS global y rutas de apartamentos sin middleware de autenticación                               | Autenticación de aplicación y autorización por usuario                                                    |
| Alta      | SSRF en importación manual                      | El servidor acepta una URL que empiece por <code>http</code> y ejecuta <code>fetch(url)</code>    | Lista de protocolos, resolución DNS/IP segura, bloqueo de redes privadas, redirecciones y límites         |
| Alta      | Estado <code>sent</code> falso                  | El bloque normal y el <code>catch</code> persisten <code>sent</code> incluso sin envío confirmado | Cambiar el estado solo tras respuesta válida de Gmail y registrar el error                                |
| Alta      | Datos personales versionados y expuestos        | <code>data.json</code> y <code>GET /api/apartments</code>                                         | Almacén privado, minimización, control de acceso y retirada del histórico Git mediante encargo específico |
| Media     | Escritura no atómica                            | Se reescribe el JSON completo sin bloqueo                                                         | Base de datos persistente o reemplazo atómico con control de concurrencia                                 |
| Media     | Ausencia de controles de abuso                  | Sin rate limiting, timeouts homogéneos ni límite explícito de cuerpo                              | Límites, timeouts, cabeceras y observabilidad sin datos sensibles                                         |

## Operación reproducible observada

### Requisitos y configuración

- Node.js compatible con las dependencias declaradas; el repositorio conserva `bun.lock`, pero no fija versión de Node ni contiene CI.
- Variables referenciadas: `GEMINI_API_KEY`, `OAuth_Client_ID`, `NODE_ENV` y `DISABLE_HMR`.
- Los valores reales deben permanecer fuera de Git. `.env.example` solo sirve de inventario y no demuestra una configuración desplegada.

### Construcción y ejecución local documentadas

```
npm install
npm run build
npm run start
```

`npm run build` genera el frontend Vite y `dist/server.cjs`; `npm run start` ejecuta el servidor empaquetado en el puerto `3000`. Estos comandos se derivan de `package.json`, pero la prueba previa se realizó sobre una copia temporal y sin invocar servicios externos.



## Limitaciones de despliegue

No existen Dockerfile, manifiesto Cloud Run, healthcheck, volumen persistente, migración ni rollback versionados. Antes de trasladar PULA a DigitalOcean deben definirse imagen reproducible, almacenamiento, secretos, autenticación, proxy/TLS, backup/restore, monitorización y pruebas.

## Riesgos y pendientes reales

### Prioridad alta

- Proteger lecturas y mutaciones con autenticación y autorización.
- Bloquear SSRF en la importación manual.
- Sacar `data.json` y los contactos del repositorio; tratar los emails como datos personales.
- No marcar `sent` tras error o ausencia de Gmail.
- Definir persistencia compatible con Cloud Run y operaciones concurrentes.
- Aportar un despliegue reproducible que vincule imagen y runtime con un SHA Git.

### Prioridad media

- Usar bloqueo y reemplazo atómico en vez de escrituras síncronas directas.
- Restringir CORS, limitar cuerpos y cargas, añadir rate limiting y cabeceras.
- Validar HTTP del scraper, fijar timeouts y evitar logs OAuth detallados.
- Añadir tests unitarios, API, scraper con fixtures, Gmail simulado y CI.
- Elegir y fijar un único gestor de dependencias y lockfile.

### Deriva documental

- Arquitectura: React 18; manifiesto: React 19.
- Documentación: scraping ilimitado; código: cinco páginas y 40 enlaces.
- `AGENTS.md` reserva Gmail y URL manual para v2.0; ambos ya están en `main`.
- La etiqueta y los artefactos ErasmusHomes están depositados en el historial de PULA pese a pertenecer al repositorio independiente `ratienza/ErasmusHomes`.
- Regla visual: prohíbe `Ver descripción`; el scraper lo asigna a `size`.
- Cloud Run y Nginx declarados sin Dockerfile ni configuración reproducible.
- La automatización Python usa mock, OAuth local y `TEST_MODE = True`; no pertenece al runtime Express demostrado.

## Pruebas y límites

-  Checkout Nexus limpio y sincronizado.
-  Instalación temporal, `tsc --noEmit` y build Vite/esbuild.
-  No existe suite automatizada.
-  No se llamó a SCPU, Gemini, Gmail, Firebase ni datos vivos.
-  Cloud Run no fue inspeccionado ni validado.
-  ErasmusHomes solo se consultó para confirmar la separación; no fue auditado.
-  No se ejecutaron los scripts `test-*`: son comprobaciones exploratorias con acceso a datos o servicios, no una suite aislada.

## Operación futura y traslado

Antes de DigitalOcean se necesitan runtime reproducible, almacenamiento persistente, secretos fuera de Git, autenticación, backup/restore, healthcheck y tests. No deben reutilizarse datos o credenciales de Cloud Run implícitamente.

`Checkout ≠ Runtime` y `Tarjeta App Launch ≠ Runtime local`: PULA no se desplegó en Nexus ni en DigitalOcean; ambos launchers enlazan la producción pública existente. El runtime público no se tocó.

# ErasmusHomes · Control del MVP

Objetivo diciembre

## Piloto comercial público antes del 20 de diciembre de 2026

Sincronizado

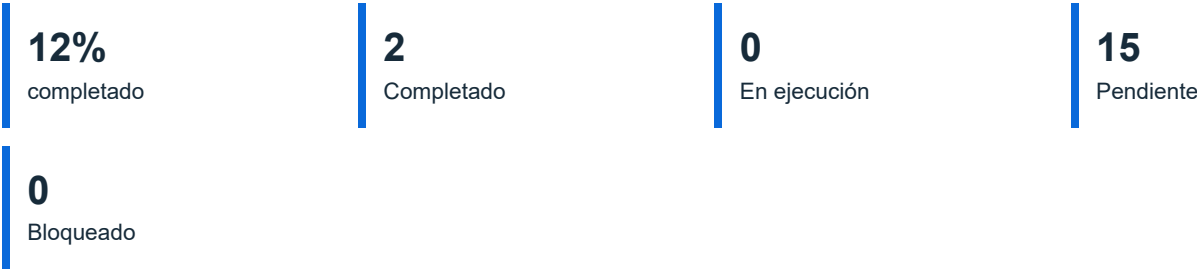
SHA ErasmusHomes main  
fe867b64f53febfe7d203718b03c2e5dff31169c  
Última sincronización  
2026-08-23T08:50:14+02:00

### Abrir roadmap acordado (DOCX)

Archivo:  
Roadmap\_ErasmusHomes\_MVP\_Diciembr  
e\_2026.docx  
Fecha: 2026-08-22

Git: fe867b64 · [ver fuente](#)  
[versionada](#)

SHA-256: f4e6be71905f7ed234ebd88a84  
d9b482b6b464b7fdd7bc4ec7241c29d5cbf  
36b



## Próximo gate

W-2026-08-31 · Supply y validación  
Retirar toda ciudad sin tres fuentes viables; no forzar scraper.

## Esta semana

No hay tareas en ejecución.

# Kanban

Completado · 2

En ejecución · 0

Pendiente · 15

Bloqueado · 0

EH-001

Consolidar la fuente de verdad

Objetivo: Consolidar fuentes, decisiones y alcance en Git y sincronizar Nexus.

Terminado: PR fusionado en main y GitHub, local y Nexus en el mismo SHA.

Riesgos: Divergencia histórica de PDFs ya documentada

Evidencia

EH-002

Roadmap canónico y panel Replicant Lab

Objetivo: Publicar roadmap estructurado y panel interno generado sin duplicar el estado.

Terminado: PRs validados, panel derivado y sincronización documentada sin nuevo servicio.

Riesgos: Integración coordinada entre tres repositorios

Evidencia

W-2026-08-31

Supply y validación

Objetivo: Evaluar hasta tres ciudades, fuentes, permisos, calidad, volumen y cinco entrevistas.

Terminado: Ficha por ciudad y decisión de piloto con tres fuentes viables por ciudad elegida.

Riesgos: Fuentes insuficientes o no permitidas

Evidencia pendiente

W-2026-09-07

Arquitectura y confianza

Objetivo: Definir datos, estados, eventos, permisos, privacidad, retención y Gmail fallback.

Terminado: ADR técnico, modelo de datos y threat checklist aceptados.

Riesgos: OAuth restringido, Sobrearquitectura

Evidencia pendiente

W-2026-09-14

UX móvil y prototipo

Objetivo: Diseñar onboarding, hoy, resultados, detalle, shortlist, contacto y seguimiento a 360-430 px.

**Terminado:** Prototipo navegable probado con 3-5 personas.

**Riesgos:** Flujos contemplativos sin acción

Evidencia pendiente

W-2026-09-21

#### Base PWA

**Objetivo:** Preparar CI, entornos, auth, perfil, PWA instalable, diseño y observabilidad mínima.

**Terminado:** La app abre en móvil y persiste el perfil de un usuario.

**Riesgos:** Incompatibilidad móvil o persistencia

Evidencia pendiente

W-2026-09-28

#### Ingesta y listings

**Objetivo:** Integrar primeras fuentes permitidas con normalización, deduplicación y origen.

**Terminado:** Listings reales o dataset trazable disponible en una ciudad.

**Riesgos:** Fuente bloqueada, Datos duplicados

Evidencia pendiente

W-2026-10-05

#### Búsqueda y selección

**Objetivo:** Implementar filtros, matching explicable, shortlist y estados.

**Terminado:** Un usuario obtiene una shortlist útil en menos de cinco minutos.

**Riesgos:** Matching opaco o sin evidencia

Evidencia pendiente

W-2026-10-12

#### Contacto seguro

**Objetivo:** Crear plantillas multilingües editables, consentimiento y derivación controlada.

**Terminado:** Mensaje contextual listo y acción auditada.

**Riesgos:** Spam, Mensaje incorrecto

Evidencia pendiente

W-2026-10-19

#### Seguimiento

**Objetivo:** Añadir estados, recordatorios, forwarding o vinculación de hilos y pendientes.

**Terminado:** Dashboard de qué hacer hoy operativo.

**Riesgos:** Auditoría OAuth retrasa el piloto

Evidencia pendiente

W-2026-10-26

#### Conversación y alertas

**Objetivo:** Resumir hilos, extraer peticiones y sugerir alertas y siguiente acción.

**Terminado:** Tres conversaciones de prueba interpretadas correctamente.

**Riesgos:** Interpretación errónea de IA

Evidencia pendiente

W-2026-11-02

#### Vertical slice

**Objetivo:** Completar buscar, seleccionar, contactar y seguir con datos reales.

**Terminado:** Demo móvil reproducible de punta a punta.

**Riesgos:** Bloqueantes P0/P1

Evidencia pendiente

W-2026-11-09

#### Calidad y cumplimiento

**Objetivo:** QA móvil, accesibilidad, seguridad, privacidad, logs, errores y recuperación.

**Terminado:** Checklist de release y registro de riesgos aceptados.

**Riesgos:** Privacidad, Accesibilidad, Recuperación incompleta

Evidencia pendiente

W-2026-11-16

#### Piloto cerrado

**Objetivo:** Incorporar primeros usuarios, prestar soporte ligero y medir activación/contacto.

**Terminado:** Primeros usuarios reales y panel de métricas.

**Riesgos:** Soporte fragmentado, Muestra insuficiente

Evidencia pendiente

W-2026-11-23

#### Corrección por evidencia

**Objetivo:** Resolver los tres mayores frenos observados y validar fuentes y mensajes.

**Terminado:** Release candidate pública basada en evidencia.

**Riesgos:** Cambios sin evidencia, Scope creep

Evidencia pendiente

W-2026-11-30

Lanzamiento piloto

**Objetivo:** Publicar PWA, monitorizar, soportar y comunicar cobertura honestamente.

**Terminado:** Piloto comercial público estable y observable.

**Riesgos:** Inestabilidad, Cobertura mal comunicada

Evidencia pendiente

W-2026-12-07

Estabilización y decisión de enero

**Objetivo:** Corregir incidencias, medir embudo, entrevistar y decidir el backlog de enero.

**Terminado:** Informe de piloto y backlog 2027 priorizado.

**Riesgos:** Escalado prematuro

Evidencia pendiente

Línea temporal hasta el 20 de diciembre

|                         |                                        |            |
|-------------------------|----------------------------------------|------------|
| 2026-08-22 - 2026-08-30 | Consolidar la fuente de verdad         | Completado |
| 2026-08-22 - 2026-08-30 | Roadmap canónico y panel Replicant Lab | Completado |

|                         |                                           |           |
|-------------------------|-------------------------------------------|-----------|
| 2026-08-31 - 2026-09-06 | <b>Supply y validación</b>                | Pendiente |
| 2026-09-07 - 2026-09-13 | <b>Arquitectura y confianza</b>           | Pendiente |
| 2026-09-14 - 2026-09-20 | <b>UX móvil y prototipo</b>               | Pendiente |
| 2026-09-21 - 2026-09-27 | <b>Base PWA</b>                           | Pendiente |
| 2026-09-28 - 2026-10-04 | <b>Ingesta y listings</b>                 | Pendiente |
| 2026-10-05 - 2026-10-11 | <b>Búsqueda y selección</b>               | Pendiente |
| 2026-10-12 - 2026-10-18 | <b>Contacto seguro</b>                    | Pendiente |
| 2026-10-19 - 2026-10-25 | <b>Seguimiento</b>                        | Pendiente |
| 2026-10-26 - 2026-11-01 | <b>Conversación y alertas</b>             | Pendiente |
| 2026-11-02 - 2026-11-08 | <b>Vertical slice</b>                     | Pendiente |
| 2026-11-09 - 2026-11-15 | <b>Calidad y cumplimiento</b>             | Pendiente |
| 2026-11-16 - 2026-11-22 | <b>Piloto cerrado</b>                     | Pendiente |
| 2026-11-23 - 2026-11-29 | <b>Corrección por evidencia</b>           | Pendiente |
| 2026-11-30 - 2026-12-06 | <b>Lanzamiento piloto</b>                 | Pendiente |
| 2026-12-07 - 2026-12-20 | <b>Estabilización y decisión de enero</b> | Pendiente |

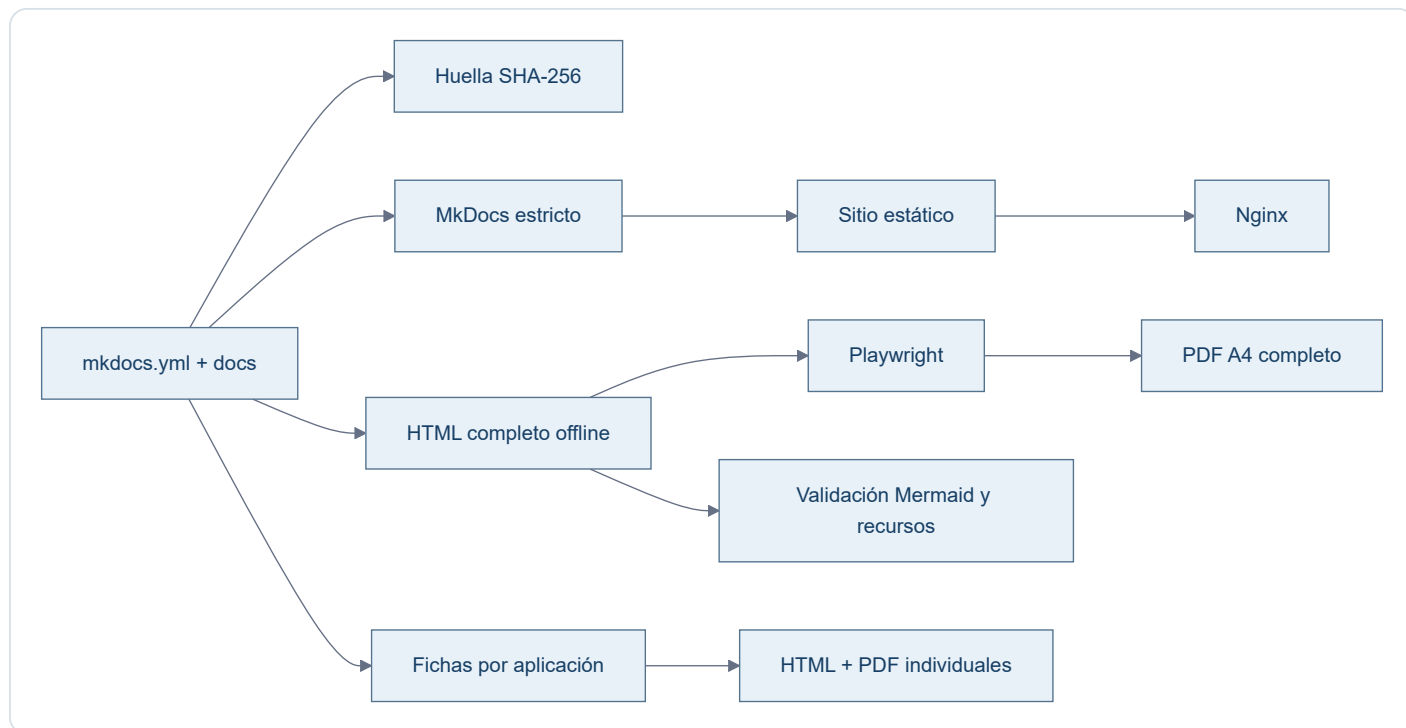
La fuente canónica es `ratienza/ErasmusHomes/docs/board/roadmap.yaml` . Esta página es un artefacto derivado.



# Autodocumentación

## Estado implementado

La documentación mantiene una fuente humana canónica ( `mkdocs.yml` , `docs/` y recursos ) y una cadena reproducible para construir el sitio y sus artefactos derivados.



## Herramientas versionadas

- `scripts/docs_pipeline.py` : orquesta construcción, generación, sincronía y validaciones estructurales.
- `scripts/render_portables.mjs` : renderiza Mermaid en Chromium y produce PDF y evidencias.
- `scripts/render_app_portable.mjs` : valida escritorio/móvil y genera cada PDF individual.
- `scripts/portable.css` : presentación compartida por HTML offline y PDF.
- `requirements-docs.txt` : dependencias Python exactas.
- `package.json` y `pnpm-lock.yaml` : Mermaid, Playwright y árbol Node fijados.
- `docs/javascripts/mermaid.min.js` : runtime derivado de la versión Mermaid fijada y comprobado contra `node_modules` .

## Huella y reproducibilidad

La huella incluye configuración MkDocs, páginas en `nav` , recursos y herramientas que afectan a los portables. Los HTML deben coincidir byte a byte con la salida esperada. Los PDF se comprueban mediante esa huella visible, cobertura textual y equivalencia de paginación con una regeneración de control.

## Temporales

Todos los resultados intermedios viven bajo `.build/docs-pipeline/` . Entornos virtuales, `node_modules` , navegadores, cachés, capturas y reportes no entran en Git.

## Flujo de actualización

1. modificar exclusivamente fuentes MkDocs;
2. ejecutar `python scripts/docs_pipeline.py generate ;`
3. revisar el sitio, HTML, PDF y capturas;
4. ejecutar `python scripts/docs_pipeline.py check ;`
5. revisar el diff completo;
6. commit, push y Pull Request;
7. tras merge, reconstruir el servicio estático en Nexus mediante Compose;
8. validar publicación y descargas en Nexus como un estado separado.

# Decisiones arquitectónicas

Registro corto de decisiones que no conviene redescubrir.

| Decisión                                       | Motivo                                                                                                                                                         |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nexus usa IP fija <code>.220</code>            | Identidad estable del servidor local                                                                                                                           |
| Replicant usa <code>.200</code>                | Identidad estable del host físico                                                                                                                              |
| Alias locales sin servidor DNS                 | Replicant resuelve <code>nexus → 192.168.18.220</code> y <code>replicant → 192.168.18.200</code> mediante <code>hosts</code> ; no se añade infraestructura DNS |
| Docker como estándar                           | Mantener limpio el host Ubuntu                                                                                                                                 |
| GitHub como source of truth                    | Reproducibilidad e histórico                                                                                                                                   |
| Datos y secretos fuera de Git                  | Seguridad y separación de responsabilidades                                                                                                                    |
| UFW restringe SSH a LAN                        | Reducir superficie de exposición                                                                                                                               |
| No usar Portainer/Webmin/Cockpit por defecto   | Evitar complejidad innecesaria                                                                                                                                 |
| Ramas por cambio lógico                        | Facilitar revisión y mantener <code>main</code> estable                                                                                                        |
| MkDocs es la fuente documental canónica        | <code>mkdocs.yml</code> , <code>docs/</code> y sus recursos definen contenido, estructura y presentación                                                       |
| HTML/PDF son artefactos derivados              | Deben sincronizarse mediante un proceso reproducible; nunca prevalecen sobre MkDocs                                                                            |
| Nexus sirve una imagen estática Nginx          | <code>Dockerfile</code> , <code>Compose</code> y <code>CI</code> comparten build MkDocs estricto; el despliegue requiere reconstrucción                        |
| Validación por entorno                         | Git, prueba local, Nexus y producción son estados distintos y no se extrapolan                                                                                 |
| Dependencias documentales fijadas              | Python/Node, MkDocs, Mermaid y Playwright se resuelven desde lockfiles versionados                                                                             |
| Huella SHA-256 para portables                  | Evita timestamps variables y permite demostrar sincronía inequívoca con las fuentes                                                                            |
| PDF tratado como binario                       | <code>.gitattributes</code> anula <code>diff=astextplain</code> y conversiones CRLF de Git para Windows                                                        |
| Rutas portables únicas                         | HTML y PDF viven exclusivamente en <code>docs/downloads/</code>                                                                                                |
| App Launch usa puerto <code>80</code> en Nexus | Permite <code>http://nexus/</code> sin puerto explícito; <code>8080</code> queda libre                                                                         |
| App Launch selecciona catálogo al desplegar    | Una presentación común y un único <code>apps.json</code> activo por host evitan filtrar enlaces internos                                                       |
| Checkouts no equivalen a servicios             | CV y Control de Red pueden existir en disco de Nexus sin declararse desplegados                                                                                |
| Fichas técnicas derivadas por aplicación       | Cada aplicación tiene HTML/PDF generado desde su Markdown canónico                                                                                             |
| Salones AV liga <code>8081</code> a la IP LAN  | <code>192.168.18.220:8081:80</code> evita publicar el servicio en todas las interfaces y queda versionado desde <code>8c0bc08</code>                           |
| App Launch es navegación, no runtime           | Una tarjeta puede apuntar a servicios locales o remotos sin trasladar su ejecución al host del launchpad                                                       |
| Consumos Cupra usa Cloud Run                   | La cadena canónica es GitHub → Cloud Build → Artifact Registry → Cloud Run; DigitalOcean y Nexus no son su producción                                          |
| CV usa Firebase Hosting                        | La producción observada pertenece al proyecto <code>replicant-lab</code> ; el Cloud Run placeholder no es producción                                           |

# Change Log

Acceso estable al histórico diario de Raul Lab.

- [21 de agosto de 2026](#)
- [13 de agosto de 2026](#)
- [9 de agosto de 2026](#)
- [8 de agosto de 2026](#)

Las entradas se ordenan de la fecha más reciente a la más antigua.

# Change Log · 21 de agosto de 2026

## Auditoría y documentación de PULA

- Clonado `ratienza/Pula` en Nexus como checkout de consulta, sin convertirlo en runtime.
- Auditado `main` ( `ff8165c` ) como POC privada de búsqueda de apartamentos SCPU en Pula, Croacia.
- Incorporados arquitectura, manual funcional, API REST, riesgos y pendientes propios de PULA.
- Compilados TypeScript y el build Vite/esbuild sobre una copia temporal; PULA permaneció limpio.
- Registrada la ausencia de tests automatizados y de trazabilidad reproducible de Cloud Run.

## Corrección de separación de proyectos

- Excluidos de PULA el concepto ErasmusHomes, el ejemplo Bolonia/UNIBO, Smart Deposit Audit y el dossier ReportLab.
- Confirmado que su ubicación canónica es el repositorio independiente `ratienza/ErasmusHomes` .
- Clasificados la etiqueta y los artefactos ErasmusHomes del historial de PULA como contaminación histórica, no como versión o capacidad de PULA.
- ErasmusHomes no fue auditado ni registrado todavía como aplicación funcional.

## Documentación integral y portable externo de PULA

- Completadas la especificación de API, persistencia, seguridad y operación reproducible a partir del código de `main` .
- Añadidos el diagrama de arquitectura y el aviso visible que separa PULA de ErasmusHomes.
- Incorporados los pendientes reales de PULA al backlog del laboratorio sin presentarlos como funciones validadas.
- Regenerados desde MkDocs los portables HTML/PDF de PULA y el dossier global mediante el pipeline reproducible.

No se modificaron PULA, ErasmusHomes, Cloud Run, Firebase, Gmail, Gemini, SCPU, DigitalOcean, datos vivos ni credenciales.

## Publicación de PULA en App Launch

- Añadida PULA a los catálogos público y Nexus de `ratienza/Apps_Lauch` mediante el squash `2d265ee` .
- Desplegados exclusivamente ambos launchers desde la fuente versionada y verificada su coincidencia exacta con cada `apps.json` .
- Validada con HTTP `200` la URL pública `https://pula-erasmus-housing-automator.ai.studio/` y su destino idéntico desde ambos catálogos.
- Nexus mantuvo estable el contenedor `app-launch` ; DigitalOcean mantuvo Nginx activo.
- Las tarjetas son enlaces: PULA no se trasladó ni se ejecuta en Nexus o DigitalOcean.

No se modificaron PULA, ErasmusHomes, el runtime público de Google, Firebase, Gmail, Gemini, SCPU, datos vivos, secretos ni otros servicios.

# Change Log · 13 de agosto de 2026

## Auditoría global y App Launch

- Confirmada la implementación multientorno de App Launch en `ced6e14`.
- Validados DigitalOcean por HTTP/HTTPS y Nexus por el puerto `80`.
- Confirmado `8080` libre y `8081-8083` sanos.
- Verificada la separación exacta de catálogos y la ausencia de referencias Nexus en DigitalOcean.

## Inventario de aplicaciones

- Auditados repositorio, SHA, arquitectura, despliegue y estado real de ocho aplicaciones.
- Incorporadas fichas individuales para App Launch, Consumos Cupra, CV y Control de Red.
- Ampliadas las fichas de Salones AV, Reserva-Pistas-UTP y Cartera Estratégica.
- Incorporada una ficha individual de Replicant Lab, separada del dossier global.
- Diferenciados explícitamente servicios, checkouts de solo lectura, producción cloud y MVP local.

## Evidencias y pendientes

- Reservas: 21 pruebas correctas.
- Consumos: TypeScript y build correctos.
- CV: build y PDF correctos.
- Salones: enlaces locales y rutas publicadas correctos.
- Control de Red: sintaxis PowerShell correcta sin escaneo.
- Cartera: 369 pruebas recogidas; ejecución parcial sin fallos hasta el límite temporal, no declarada como suite completa.
- Conservadas como pendientes la deriva Compose de Salones, la trazabilidad de despliegue de Consumos, la configuración Cloud del CV, los datos versionados de Control de Red y la política global de backups.

No se modificaron CV/Firebase/Cloud Run, Cartera, red, firewall, DNS, certificados, secretos ni datos privados.

## Fase 2A · Remediación de Salones AV

- Resuelta la deriva de `compose.yml` mediante el PR `salones-av-valencia-palace#2`.
- Versionado el bind LAN `192.168.18.220:8081:80` en `main` (`8c0bc08`).
- Reconciliado Nexus preservando y comparando el cambio local antes del avance rápido.
- Verificados Compose, blob Git, checkout limpio, bind efectivo y las siete rutas con respuesta `200`.
- No se modificaron contenido AV, fuentes documentales, datos, otros contenedores ni producción DigitalOcean.

## Consolidación final pre-Cartera

- Cerradas documentalmente las fases 2A, 2B y 2C sin reabrir remediaciones.
- Consumos Cupra queda trazado desde `9f66a368` hasta Cloud Run `cosnumos-cupra-00009-pon`; DigitalOcean y Nexus no son su runtime.
- El CV queda identificado en Firebase Hosting del proyecto `replicant-lab`, operativo en `cv.raulatienza.com`, con deuda aceptada POST-CARTERA.
- App Launch queda definido como catálogo y navegación: una tarjeta no demuestra runtime local.
- La navegación histórica se reduce a hitos y las pruebas detalladas permanecen en las fichas de las ocho aplicaciones.

# 9 de agosto de 2026

Change Log diario de Raul Lab. Reúne los cambios documentales, técnicos y operativos de la fecha, del hito más reciente al más antiguo.

## Cierre integral de Reserva-Pistas-UTP · validación final 10/08/2026

- Los [PR #4](#), [#5](#) y [#6](#) integraron mediante squash el motor de sincronización, la corrección del usuario de runtime y la guía de despliegue validada. El borrador #1 quedó cerrado al ser sustituido.
- La aplicación dejó de exponer contraseñas y de incluir credenciales en las tareas. El almacenamiento incorpora validación estricta, IDs estables, bloqueo entre procesos, escritura atómica, copias, restauración, ledger y detección de conflictos.
- El panel permite vista previa y confirmación explícita en ambas direcciones, evita dobles ejecuciones y presenta conflictos solo con campos no sensibles. La transferencia se realiza desde Nexus mediante clave dedicada y comando SSH forzado en DigitalOcean.
- Se superaron 21 pruebas, compilación Python, sintaxis JavaScript, configuración y construcción Docker, controles HTTP y revisión de logs.
- Antes de intervenir se conservaron copias de seguridad; los datos funcionales y los ficheros locales de credenciales y notificaciones de producción mantuvieron su integridad.
- Se replicaron de DigitalOcean a Nexus 18 registros —12 cancelados y 6 reservados—, sin tareas activas, duplicados ni conflictos. Después, ambas vistas previas convergieron en 18 elementos sin cambios y el documento normalizado compartió el hash `db9a806429479d9e83c83724ca67b5e072ca2ab29b94fbde9dbd19475342dc09`.
- Nexus quedó operativo en `192.168.18.220:8083` y DigitalOcean en `https://app.raulatienza.com/padel/`; no se alteraron DNS, certificados, puertos ni la topología pública. El [issue #2](#) quedó cerrado con evidencias.

## Corrección final de experiencia documental · PR #11

- El [PR #11](#) integra la corrección final de cronología, visión transversal de repositorios y experiencia portable.
- La sección **Cambios** pasa a presentarse como **Change Log**, con una cronología única, fechas visibles y contexto técnico y operativo.
- **Pendientes** amplía su alcance a los ocho repositorios vinculados a Raul Lab comprobados en GitHub; `ratienza/python` queda excluido expresamente.
- El HTML independiente recupera una experiencia multipágina dentro de un único fichero autocontenido: índice persistente, una vista documental activa, anterior/siguiente, fragmentos directos, historial atrás/adelante, búsqueda local y adaptación móvil.
- El PDF mantiene su modelo editorial completo; HTML y PDF continúan generándose desde la fuente MkDocs canónica y no desde exportaciones antiguas.

## 18:55 · Estado real de Nexus registrado · PR #10

- El [PR #10](#) integró la evidencia de despliegue y cerró los estados que todavía figuraban como pendientes.
- `main` quedó en `cd09dec4cb083f7a761e09648e23404cfd35da0c`.
- Nexus quedó sincronizado, con Nginx estático estable en `192.168.18.220:8082`, cinco Mermaid y descargas verificadas.

## 18:39 · Pipeline reproducible y runtime estático · PR #9

- El [PR #9](#) integró el pipeline reproducible de documentación.
- MkDocs —`mkdocs.yml`, `docs/` y sus recursos— quedó reafirmado como fuente canónica de contenido, estructura y presentación.
- Docker, Compose y CI convergieron en `mkdocs build --strict` y una imagen final Nginx; desaparecieron `mkdocs serve`, los `bind mounts` y el runtime de desarrollo en Nexus.
- Mermaid 11.16.1 pasó a servirse localmente, sin CDN, y los cinco diagramas quedaron disponibles también offline.
- Se generaron un HTML autocontenido completo y un PDF A4 completo desde las 27 páginas de navegación.
- Las rutas canónicas quedaron fijadas en `docs/downloads/Replicant-Lab.html` y `docs/downloads/Replicant-Lab.pdf`; se eliminó el HTML duplicado obsoleto.
- La validación local incluyó MkDocs estricto, enlaces, recursos, Docker, escritorio, móvil, HTML `file://`, PDF y ausencia de CDN, localhost, WebSocket y recarga en vivo.
- El squash resultante fue `f457c57b472515afe46025078c7342dd5a75a7f1`.

## Despliegue y validación real en Nexus

- Se actualizó exclusivamente `/opt/apps/infra-replicant-lab` mediante avance rápido y `docker compose up -d --build`.
- El contenedor `infra-replicant-docs` quedó activo con Nginx `1.27.4`, reinicios `0` y publicación `192.168.18.220:8082 → 80`.
- Se comprobaron navegación, recursos, presentación de escritorio y móvil, cinco Mermaid, HTML offline y PDF completo.
- Las descargas reales coincidieron byte a byte con Git:
- HTML: `17a2746d2e11e26d0302a57633c5b1b05119348d8e4a4629f00d67ab8cfce6a1`.
- PDF: `028a135d7645912569c5d6c6c13b7c885b250084e3b269add7d10f89f627e13b`.
- Los logs finales no mostraron errores, `404`, recarga en vivo, WebSocket ni dependencias esenciales externas.

### 17:11 · Reconciliación y consolidación documental · PR #8

- El [PR #8](#) reconcilió las fuentes MkDocs con el estado comprobado de GitHub y Nexus.
- Se separaron con claridad Replicant/Hyper-V, Nexus, DigitalOcean y los repositorios propios de cada aplicación.
- README, navegación, operación, despliegue, aplicaciones y pendientes quedaron alineados; el procedimiento de Nexus distinguió primer arranque de actualización ordinaria.
- El squash resultante fue `9c31728bfb0a4399dc736bea59f7ed50ed480bf3`.

### 16:07 · Contrato operativo raíz · PR #7

- El [PR #7](#) incorporó `AGENTS.md`.
- El contrato fijó la jerarquía entre verdad técnica versionada, documentación MkDocs canónica y artefactos HTML/PDF derivados.
- También formalizó separación de entornos, protección de datos, flujo rama–pruebas–PR–merge y prohibición de desplegar o modificar producción sin autorización.
- El squash resultante fue `01d859fac7b7907ae92cf65bda2ad50cfb0e738f`.



## 8 de agosto de 2026

Change Log diario de Raul Lab. Se conserva íntegramente el histórico previamente registrado para esta fecha.

- Replicant queda con IP fija `192.168.18.200` .
- Nexus queda con IP fija `192.168.18.220` mediante Netplan.
- SSH por clave consolidado.
- UFW activado con SSH permitido solo desde la LAN.
- Se verifican Docker, Git y actualizaciones automáticas.
- Salones AV queda accesible desde la LAN en `192.168.18.220:8081` .
- Replicant Lab queda publicado en Nexus por `192.168.18.220:8082` .
- Reserva-Pistas-UTP pasa de un staging manual a un despliegue Docker Compose completo en Nexus: backend Python + Nginx en red privada Docker, acceso LAN por `192.168.18.220:8083` .
- El backend de Reserva-Pistas se hace configurable mediante `APP_HOST` , `APP_PORT` y `APP_DATA_DIR` , manteniendo valores por defecto compatibles con el despliegue anterior.
- La persistencia de Reserva-Pistas se separa en `/opt/data/reserva-pistas` y se monta como `/app/data` .
- El backend Docker se ejecuta como UID/GID `1000:1000` y ambos servicios usan `restart: unless-stopped` .
- Se valida HTTP `200` , persistencia tras recreación de contenedores y recuperación automática tras reinicio completo de Nexus.
- UFW permite `8083/tcp` exclusivamente desde `192.168.18.0/24` .
- Se formaliza la configuración Nexus en `ratienza/Reserva-Pistas-UTP` y se integra en `main` mediante PR #3.
- DigitalOcean permanece sin cambios y continúa como producción de Reserva-Pistas-UTP.
- Queda pendiente sincronizar a Nexus el histórico/estado vigente de `tasks.local.json` desde DigitalOcean, verificando antes que no existan tareas activas que puedan duplicarse.
- La portada documental incorpora accesos de descarga al HTML autónomo y al PDF de Replicant Lab.
- DNS local y backups se mantienen pendientes conscientemente.
- Se consolida `infra-replicant-lab` como documentación viva.